MainGUI.java

```
import javax.swing.*;
import java.awt.*;
import java.util.HashMap;

public class MainGUI {
    private JFrame frame;
    private JPasswordField passwordField;
    private JCheckBox showPasswordCheck;
    private JProgressBar strengthBar;
    private JLabel strengthLabel;
    private JPanel rulesPanel;
    private JButton checkBtn, generateBtn, saveBtn;
    private JTextField lengthField;
    private JCheckBox includeNumbersCheck, includeSymbolsCheck;

    private HashMap<String, Integer> passwordHistory;
    private PasswordGenerator generator;

    public MainGUI() {
        passwordHistory = new HashMap<>();
        generator = new PasswordGenerator();
        initUI();
    }

    private void initUI() {
        frame = new JFrame("Password Strength Checker");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout(10, 10));

        JLabel title = new JLabel("Password Strength Checker", SwingConstants.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 18));
        frame.add(title, BorderLayout.NORTH);

        JPanel centerPanel = new JPanel();
        centerPanel.setLayout(new BoxLayout(centerPanel, BoxLayout.Y_AXIS));
        centerPanel.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));

        JPanel inputPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        inputPanel.add(new JLabel("Enter Password:"));
        passwordField = new JPasswordField(20);
        passwordField.setEchoChar('\u2022');
        inputPanel.add(passwordField);
        showPasswordCheck = new JCheckBox("Show");
        showPasswordCheck.addActionListener(e -> {
            if (showPasswordCheck.isSelected()) {
                passwordField.setEchoChar((char) 0);
            } else {
                passwordField.setEchoChar('\u2022');
            }
        });
        inputPanel.add(showPasswordCheck);
        centerPanel.add(inputPanel);

        JPanel strengthPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        strengthBar = new JProgressBar(0, 10);
        strengthBar.setPreferredSize(new Dimension(200, 25));
        strengthBar.setStringPainted(true);
        strengthPanel.add(new JLabel("Strength:"));
```

```
        strengthPanel.add(strengthBar);
        strengthLabel = new JLabel("Not checked");
        strengthPanel.add(strengthLabel);
        centerPanel.add(strengthPanel);

        rulesPanel = new JPanel();
        rulesPanel.setLayout(new BoxLayout(rulesPanel, BoxLayout.Y_AXIS));
        rulesPanel.setBorder(BorderFactory.createTitledBorder("Rules"));
        centerPanel.add(rulesPanel);

        JPanel genPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        genPanel.setBorder(BorderFactory.createTitledBorder("Generator"));
        genPanel.add(new JLabel("Length:"));
        lengthField = new JTextField(5);
        lengthField.setText("16");
        genPanel.add(lengthField);
        includeNumbersCheck = new JCheckBox("Numbers");
        includeNumbersCheck.setSelected(true);
        genPanel.add(includeNumbersCheck);
        includeSymbolsCheck = new JCheckBox("Symbols");
        includeSymbolsCheck.setSelected(true);
        genPanel.add(includeSymbolsCheck);
        centerPanel.add(genPanel);

        frame.add(centerPanel, BorderLayout.CENTER);

        JPanel btnPanel = new JPanel(new FlowLayout());
        checkBtn = new JButton("Check");
        generateBtn = new JButton("Generate");
        saveBtn = new JButton("Save");
        btnPanel.add(checkBtn);
        btnPanel.add(generateBtn);
        btnPanel.add(saveBtn);
        frame.add(btnPanel, BorderLayout.SOUTH);

        checkBtn.addActionListener(e -> checkPassword());
        generateBtn.addActionListener(e -> generatePassword());
        saveBtn.addActionListener(e -> savePassword());

        frame.setSize(450, 450);
        frame.setVisible(true);
        frame.setLocationRelativeTo(null);
    }

    private void checkPassword() {
        try {
            String password = new String(passwordField.getPassword());
            if (password == null || password.isEmpty()) {
                throw new IllegalArgumentException("Password cannot be empty");
            }

            PasswordAnalyzer analyzer = new PasswordAnalyzer(password);
            int score = analyzer.getScore();
            String strength = analyzer.getStrengthLabel();

            strengthBar.setValue(score);
            strengthLabel.setText(strength);

            if (score <= 3) {
                strengthBar.setForeground(Color.RED);
            } else if (score <= 6) {
                strengthBar.setForeground(Color.ORANGE);
```

```
            } else {
                strengthBar.setForeground(Color.GREEN);
            }

            rulesPanel.removeAll();
            for (String rule : analyzer.getRules()) {
                rulesPanel.add(new JLabel(rule));
            }
            rulesPanel.revalidate();
            rulesPanel.repaint();

            passwordHistory.put(password, score);

            PasswordPolicy weak = new WeakPolicy();
            PasswordPolicy strong = new StrongPolicy();
            boolean weakValid = weak.validate(password);
            boolean strongValid = strong.validate(password);

            String msg = "Score: " + score + "/10 (" + strength + ")\n";
            msg += "HashSet types found: " + analyzer.getTypesFound() + "\n";
            msg += "History entries: " + passwordHistory.size() + "\n";
            msg += "WeakPolicy: " + (weakValid ? "Valid" : "Invalid") + "\n";
            msg += "StrongPolicy: " + (strongValid ? "Valid" : "Invalid");

            JOptionPane.showMessageDialog(frame, msg, "Analysis Result",
JOptionPane.INFORMATION_MESSAGE);

        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(frame, e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        } finally {
            strengthBar.repaint();
        }
    }

    private void generatePassword() {
        try {
            int length = Integer.parseInt(lengthField.getText().trim());
            if (length <= 0) throw new NumberFormatException();

            generator.includeNumbers(includeNumbersCheck.isSelected());
            generator.includeSymbols(includeSymbolsCheck.isSelected());

            String generated = generator.generate(length);
            passwordField.setText(generated);
            passwordField.setEchoChar((char) 0);
            showPasswordCheck.setSelected(true);

            PasswordAnalyzer analyzer = new PasswordAnalyzer(generated);
            int score = analyzer.getScore();
            String strength = analyzer.getStrengthLabel();

            strengthBar.setValue(score);
            strengthLabel.setText(strength);

            if (score <= 3) {
                strengthBar.setForeground(Color.RED);
            } else if (score <= 6) {
                strengthBar.setForeground(Color.ORANGE);
            } else {
                strengthBar.setForeground(Color.GREEN);
            }
```

```
            rulesPanel.removeAll();
            for (String rule : analyzer.getRules()) {
                rulesPanel.add(new JLabel(rule));
            }
            rulesPanel.revalidate();
            rulesPanel.repaint();

            JOptionPane.showMessageDialog(frame,
                "Generated: " + generated + "\nStrength: " + strength + " (" + score +
"/10)",
                "Generated Password", JOptionPane.INFORMATION_MESSAGE);

        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(frame, "Invalid length. Enter a positive
number.", "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void savePassword() {
        try {
            String password = new String(passwordField.getPassword());
            if (password == null || password.isEmpty()) {
                throw new IllegalArgumentException("Nothing to save");
            }
            generator.saveToFile(password);
            JOptionPane.showMessageDialog(frame, "Password saved to
saved_passwords.txt", "Saved", JOptionPane.INFORMATION_MESSAGE);
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(frame, e.getMessage(), "Error",
JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(frame, "Save failed: " + e.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(MainGUI::new);
    }
}
```

PasswordAnalyzer.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;

public class PasswordAnalyzer {
    private String _password;
    private int _score;
    private ArrayList<String> _rules;
    private HashMap<String, Integer> _history;
    private HashSet<String> _typesFound;

    public PasswordAnalyzer(String password) {
        this._password = password;
        this._rules = new ArrayList<>();
        this._history = new HashMap<>();
```

```
        this._typesFound = new HashSet<>();
        analyze();
    }

    private void analyze() {
        _rules.clear();
        _score = 0;

        if (_password == null || _password.isEmpty()) {
            _rules.add("Password is empty");
            return;
        }

        _typesFound.clear();
        for (char c : _password.toCharArray()) {
            if (Character.isUpperCase(c)) _typesFound.add("UPPER");
            else if (Character.isLowerCase(c)) _typesFound.add("LOWER");
            else if (Character.isDigit(c)) _typesFound.add("DIGIT");
            else _typesFound.add("SPECIAL");
        }

        if (_password.length() >= 8) { _rules.add("Min 8 characters"); _score++; }
        else { _rules.add("Min 8 characters"); }

        if (_password.length() >= 12) { _rules.add("Min 12 characters"); _score++; }
        else { _rules.add("Min 12 characters"); }

        if (_password.length() >= 16) { _rules.add("Min 16 characters"); _score++; }
        else { _rules.add("Min 16 characters"); }

        if (_password.matches(".*[a-z].*")) { _rules.add("Has lowercase letter");
_score++; }
        else { _rules.add("Has lowercase letter"); }

        if (_password.matches(".*[A-Z].*")) { _rules.add("Has uppercase letter");
_score++; }
        else { _rules.add("Has uppercase letter"); }

        if (_password.matches(".*[0-9].*")) { _rules.add("Has number"); _score++; }
        else { _rules.add("Has number"); }

        if (_password.matches(".*[!@#$%^&*()_+\\-=\\[\\]{};':\"\\\\|,.<>\\/?].*")) {
            _rules.add("Has special character"); _score++;
        } else {
            _rules.add("Has special character");
        }

        if (_typesFound.size() >= 3) { _rules.add("3+ character types"); _score++; }
        else { _rules.add("3+ character types"); }

        if (_typesFound.size() >= 4) { _rules.add("All 4 character types"); _score++; }
        else { _rules.add("All 4 character types"); }

        boolean noRepeat = true;
        for (int i = 1; i < _password.length(); i++) {
            if (_password.charAt(i) == _password.charAt(i - 1)) {
                noRepeat = false;
                break;
            }
        }
        if (noRepeat) { _rules.add("No repeated consecutive chars"); _score++; }
        else { _rules.add("No repeated consecutive chars"); }
```

```
        _history.put(_password, _score);
    }

    public int getScore() { return _score; }

    public ArrayList<String> getRules() { return _rules; }

    public HashMap<String, Integer> getHistory() { return _history; }

    public HashSet<String> getTypesFound() { return _typesFound; }

    public String getStrengthLabel() {
        if (_score <= 3) return "Weak";
        if (_score <= 6) return "Medium";
        return "Strong";
    }
}
```

PasswordGenerator.java

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

public class PasswordGenerator {
    private static final String LETTERS =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    private static final String NUMBERS = "0123456789";
    private static final String SYMBOLS = "!@#$%^&*()_+-=[]{}|;:,.<>?";

    private boolean includeNumbers;
    private boolean includeSymbols;
    private Random random;

    public PasswordGenerator() {
        this.includeNumbers = true;
        this.includeSymbols = true;
        this.random = new Random();
    }

    public void includeNumbers(boolean include) { this.includeNumbers = include; }
    public void includeSymbols(boolean include) { this.includeSymbols = include; }

    public String generate(int length) {
        StringBuilder pool = new StringBuilder(LETTERS);
        if (includeNumbers) pool.append(NUMBERS);
        if (includeSymbols) pool.append(SYMBOLS);

        StringBuilder password = new StringBuilder();
        for (int i = 0; i < length; i++) {
            int index = random.nextInt(pool.length());
            password.append(pool.charAt(index));
        }
        return password.toString();
    }

    public void saveToFile(String password) throws IOException {
        FileWriter fw = null;
```

```
        try {
            fw = new FileWriter("saved_passwords.txt", true);
            fw.write(password + System.lineSeparator());
        } finally {
            if (fw != null) {
                try { fw.close(); } catch (IOException e) {
                    System.err.println("Error closing file: " + e.getMessage());
                }
            }
        }
    }
}
```

PasswordPolicy.java

```
public abstract class PasswordPolicy {
    protected int minLength;
    protected boolean requireNumbers;
    protected boolean requireSpecialChars;

    public abstract boolean validate(String password);

    public int getMinLength() { return minLength; }
    public boolean isRequireNumbers() { return requireNumbers; }
    public boolean isRequireSpecialChars() { return requireSpecialChars; }
}
```

StrongPolicy.java

```
public class StrongPolicy extends PasswordPolicy {
    public StrongPolicy() {
        this.minLength = 12;
        this.requireNumbers = true;
        this.requireSpecialChars = true;
    }

    @Override
    public boolean validate(String password) {
        if (password == null || password.length() < minLength) return false;
        if (requireNumbers && !password.matches(".*[0-9].*")) return false;
        if (requireSpecialChars &&
!password.matches(".*[!@#$%^&*()_+\\-=\\[\\]{};':\"\\\\|,.<>\\/?].*")) return false;
        if (!password.matches(".*[A-Z].*")) return false;
        return true;
    }
}
```

WeakPolicy.java

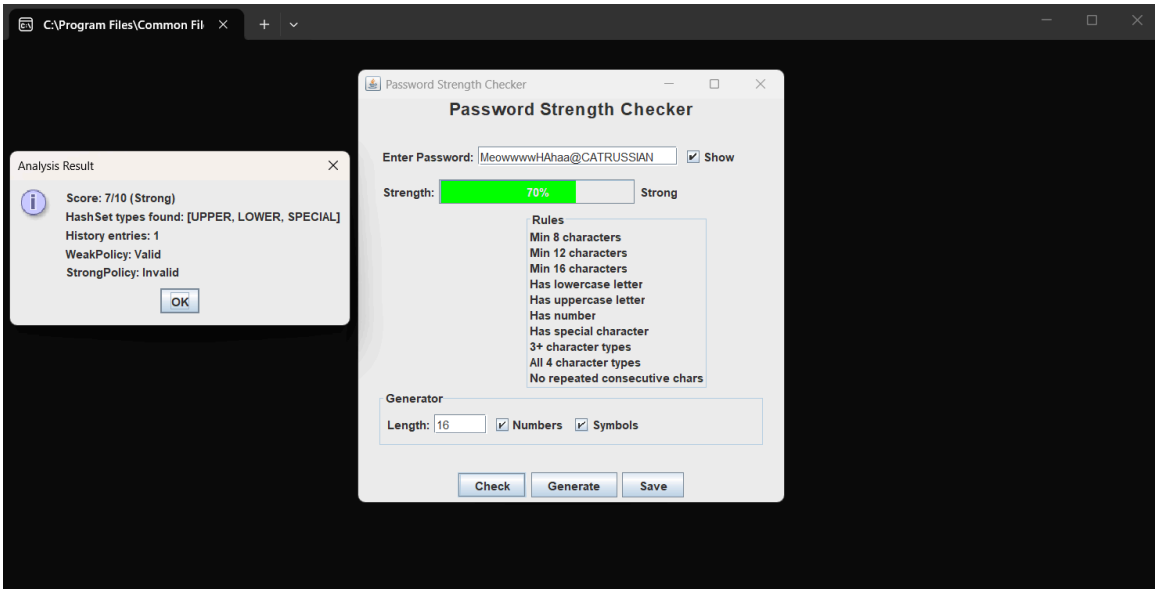
```
public class WeakPolicy extends PasswordPolicy {
    public WeakPolicy() {
        this.minLength = 6;
        this.requireNumbers = false;
        this.requireSpecialChars = false;
```

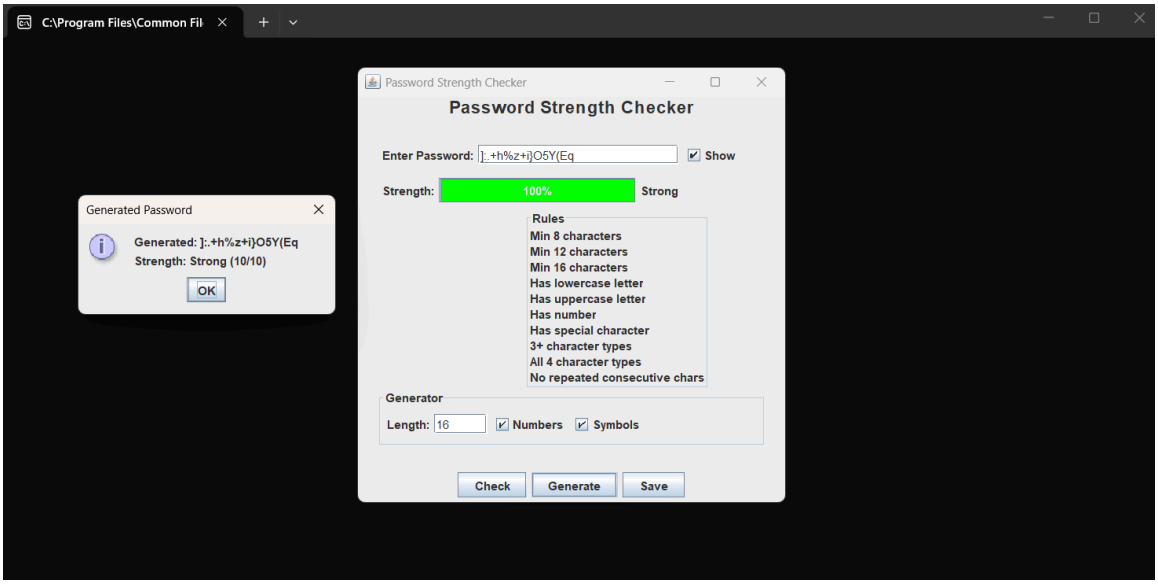
```
    }

    @Override
    public boolean validate(String password) {
        if (password == null) return false;
        return password.length() >= minLength;
    }
}
```

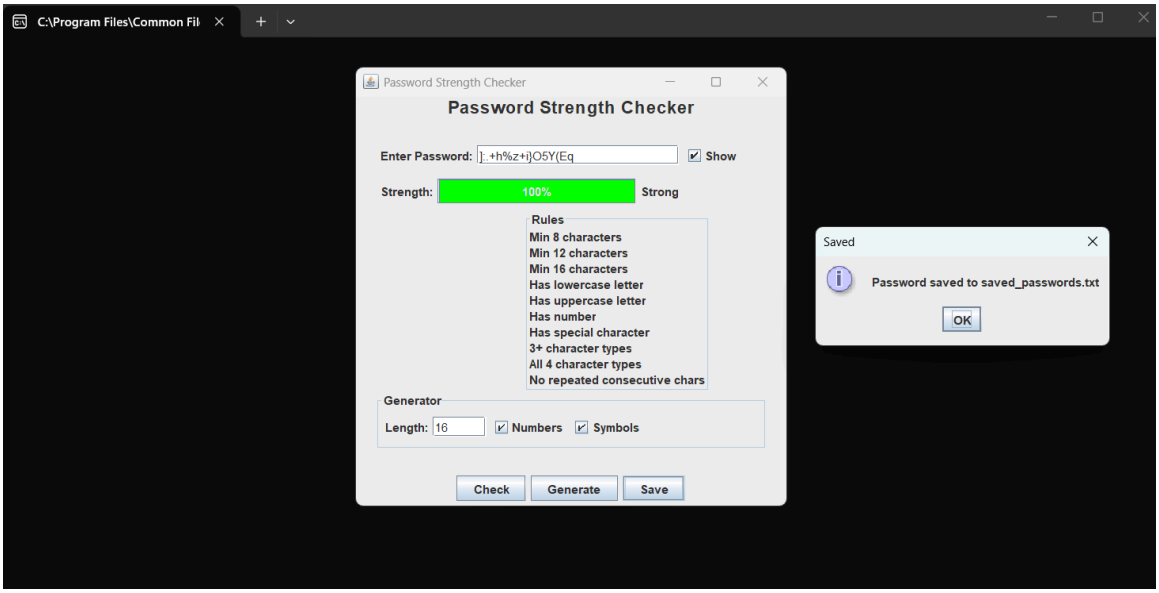
Outputs

saved_passwords.txt

```
]:.+h%z+i}O5Y(Eq
```




C:\Program Files\Common Fil
Password Strength Checker
Password Strength Checker
Enter Password:]:.+h%z+i}O5Y(Eq
Show
Strength:
100%
Strong
Rules
Min 8 characters
Min 12 characters
Min 16 characters
Has lowercase letter
Has uppercase letter
Has number
Has special character
3+ character types
All 4 character types
No repeated consecutive chars
Generator
Length: 16
Numbers
Symbols
Check
Generate
Save
Generated Password
Generated:]:.+h%z+i}O5Y(Eq
Strength: Strong (10/10)
OK


C:\Program Files\Common Fil
Password Strength Checker
Password Strength Checker
Enter Password:]:.+h%z+i}O5Y(Eq
Show
Strength:
100%
Strong
Rules
Min 8 characters
Min 12 characters
Min 16 characters
Has lowercase letter
Has uppercase letter
Has number
Has special character
3+ character types
All 4 character types
No repeated consecutive chars
Generator
Length: 16
Numbers
Symbols
Check
Generate
Save
Saved
Password saved to saved_passwords.txt
OK